Module 17 - Python Data Structures Modelling Security Data

Table of Contents

- [ ] 1. Why Data Modelling Matters
- [ ] 2. Dictionary - Quick But Fragile
- [ ] 3. Regular Class - Full Control
- [ ] 4. Dataclass - Clean Syntax
- [ ] 5. NamedTuple - Lightweight, Immutable Records
- [ ] 6. Combining Structures
- [ ] 7. When to Use Which - Decision Guide
- [ ] 8. Summary

---

1. Why Data Modelling Matters

Analogy: A SOC analyst receives an alert. Without structure, the alert is just a wall of text - every analyst reads it differently, fields get missed, comparisons become impossible. With structure, every alert has a defined set of fields: timestamp, source IP, destination, severity, category. Colleagues and tools can rely on it.

Data modelling in Python is the same idea applied to your code. When you decide **how to store a piece of information** (as a dict, a class, a dataclass, a namedtuple...), you are deciding:

- How easy it is for other functions to access the data
- Whether typos in field names get caught early or silently cause bugs
- Whether the data can change after it's created
- How much boilerplate code you have to write
- How clear the code is to the next analyst who reads it

The running example

Throughout this notebook we'll model the same object using different Python structures: **a network port scan result**.

A scan result contains:

- `target_ip` - the IP address that was scanned

- `open_ports` - list of open port numbers
- `scan_duration_ms` - how long the scan took (in milliseconds)
- `timestamp` - when the scan ran
- `analyst` - who ordered the scan

We'll see how each structure handles this data and what trade-offs each one makes.

In [ ]:
```
# Imports we'll use throughout this notebook
from typing import List, Dict, Optional, Tuple
from dataclasses import dataclass, field
from collections import namedtuple
```

---

2. Dictionary - Quick But Fragile

Analogy: A dictionary is like a sticky note. Fast to write, no fixed format, anyone can add or remove fields on the fly. Great for quick notes; terrible as an official incident report template.

The first instinct when storing structured data is often to use a `dict`. It works - but it comes with real problems at scale.

In [ ]:
```
# A dict to store one port scan result
scan_result: dict = {
    "target_ip": "10.0.0.55",
    "open_ports": [22, 80, 443, 8080],
    "scan_duration_ms": 342,
    "timestamp": "2024-01-15T08:30:00Z",
    "analyst": "j.smith",
}

# Accessing data
print(scan_result["target_ip"])
print(scan_result["open_ports"])
```

Problems with dictionaries

1. Typos fail silently (or crash at runtime, not at definition)

In [ ]:
```
# Typo in the key name - Python won't warn you until this line runs
print(scan_result["open_port"])    # KeyError: 'open_port'
# Compare: a class attribute would be caught by IDEs and mypy before running
```

In [ ]:
```
# Safe access with .get() avoids the crash but masks the bug
ports = scan_result.get("open_port")   # returns None silently
print(ports)  # None - not an error, but definitely wrong
# Now downstream code gets None instead of a list - error surfaces much later
```

2. Nothing prevents missing fields or extra fields

In [ ]:
```
# Any dict can pretend to be a scan result - no required fields enforced
incomplete_scan: dict = {
    "target_ip": "192.168.1.1"
    # missing open_ports, timestamp, analyst...
}

# Python does not complain. The error only surfaces when you try to use
# the missing fields, possibly deep inside another function.
print(incomplete_scan.get("analyst", "MISSING"))  # MISSING
```

3. No computed properties, no behaviour

In [ ]:
```
# You can't attach logic to a dict
# If you want 'is_dangerous' (True if any port is in the risky list),
# you have to compute it externally every single time
risky_ports = [21, 23, 3389, 445]
scan = {"target_ip": "10.0.0.2", "open_ports": [22, 3389]}

# Logic must live outside the data - easy to forget, easy to duplicate
is_dangerous = any(p in risky_ports for p in scan["open_ports"])
print(f"{scan['target_ip']} is dangerous: {is_dangerous}")
```

When IS a dict the right choice?

Use a dict when:

- You are writing a quick one-off script or exploring data interactively
- The structure changes dynamically (e.g., parsing an unknown log format)
- You need to serialise to JSON immediately (dicts map directly to JSON)
- The data is truly key-value with no fixed schema

Practice - Section 2

Fix: The function below tries to extract the analyst name from a scan result dict, but it will crash if the dict doesn't have the `"analyst"` key. Fix it so it returns `"unknown"` when the key is missing, without crashing.

In [ ]:
```
# Broken code - crashes when analyst key is absent
def get_analyst(scan: dict) -> str:
    return scan["analyst"]   # KeyError if key is missing


# Test (this should NOT crash)
scan_without_analyst = {"target_ip": "10.0.0.1", "open_ports": [22]}
# print(get_analyst(scan_without_analyst))  # currently crashes
```

▶ 💡 **SOLUTION**

Predict: What does the following code output? Write your answer as a comment, then run it.

In [ ]:
```
# Predict the output before running
alert: dict = {"severity": "HIGH", "source_ip": "203.0.113.5"}

alert["severity"] = "CRITICAL"
alert["ticket"] = "INC-0042"
del alert["source_ip"]

# Your prediction:
# print(alert) -> ?

print(alert)
```

▶ 💡 **SOLUTION**

---

3. Regular Class - Full Control

Analogy: If a dict is a sticky note, a class is a proper incident report form. It has defined fields, mandatory fields that must be filled in at creation, validation logic, and computed fields. It's more work to design, but much more reliable at scale.

A regular Python class (with `__init__`) gives you:

- **Required fields** - missing a required argument raises `TypeError` immediately
- **Computed attributes** - derive new data from existing fields at creation time
- **Methods** - attach behaviour (logic) directly to the data
- **Type hints** on every parameter

In [ ]:
```
from typing import List


DANGEROUS_PORTS: List[int] = [21, 23, 445, 3389]  # FTP, Telnet, SMB, RDP


class ScanResult:
    """Stores the result of a port scan against a single target host."""

    def __init__(
        self,
        target_ip: str,
        open_ports: List[int],
        scan_duration_ms: int,
        timestamp: str,
        analyst: str,
    ):
        self.target_ip = target_ip
        self.open_ports = open_ports
        self.scan_duration_ms = scan_duration_ms
```

```
        self.timestamp = timestamp
        self.analyst = analyst
        # Computed attribute: derived at creation, always consistent
        self.port_count: int = len(open_ports)
        self.has_dangerous_ports: bool = any(p in DANGEROUS_PORTS for p in open_por

    def summary(self) -> str:
        """Returns a one-line summary of this scan result."""
        risk = "[RISK]" if self.has_dangerous_ports else "[OK]"
        return (
            f"{risk} {self.target_ip} | "
            f"{self.port_count} port(s) open | "
            f"Scanned by {self.analyst} at {self.timestamp}"
        )


# Instantiate - all fields are required; missing one raises TypeError immediately
result = ScanResult(
    target_ip="10.0.0.55",
    open_ports=[22, 3389, 445],   # contains dangerous ports
    scan_duration_ms=284,
    timestamp="2024-01-15T08:30:00Z",
    analyst="j.smith",
)

print(result.target_ip)
print(result.port_count)           # computed, not passed in
print(result.has_dangerous_ports)  # computed, not passed in
print(result.summary())
```

What you get vs what you write

Feature	dict	class
Required fields enforced	No	Yes (TypeError on missing arg)
Computed attributes	No	Yes (in `__init__`)
Methods (behaviour)	No	Yes
Type hints per field	Hard	Yes, fully
Typo caught by IDE/mypy	No	Yes
Boilerplate to write	Low	Medium

The downside

Every attribute must be written **twice**: once in the `__init__` signature (`:` type annotation), and once as `self.attribute = argument` . For objects with many fields, this becomes verbose.

Practice - Section 3

Write: Create a class `SecurityAlert` to store a SOC alert with these fields:

- `alert_id` (str) - e.g. `"ALT-0099"`
- `source_ip` (str) - attacker IP
- `destination_ip` (str) - victim IP
- `severity` (str) - `"LOW"`, `"MEDIUM"`, `"HIGH"`, or `"CRITICAL"`
- `is_acknowledged` (bool) - has a SOC analyst acknowledged this alert? Default `False`

Add a computed attribute `is_critical` (bool) that is `True` when severity is `"CRITICAL"`. Add a method `acknowledge()` that sets `is_acknowledged` to `True` and prints `"Alert {alert_id} acknowledged."`

In [ ]:
```
from typing import List

# Your code here


# Test
# alert = SecurityAlert("ALT-0099", "203.0.113.5", "10.0.0.1", "CRITICAL")
# print(alert.is_critical)       # True
# print(alert.is_acknowledged)   # False
# alert.acknowledge()
# print(alert.is_acknowledged)   # True
```

▶ 💡 **SOLUTION**

---

4. Dataclass - Clean Syntax

Analogy: A dataclass is like a class where Python fills in the paperwork for you. You declare the fields and their types once. Python automatically generates `__init__`, `__repr__`, and `__eq__`. Same power as a class, but you write roughly half the code.

Use the `@dataclass` decorator from the `dataclasses` module (standard library, no install needed).

Before vs after

```
# Regular class - you write __init__ manually (field repeated twice per
attribute)
class Vulnerability:
    def __init__(self, cve_id: str, cvss_score: float, affected_system:
str):
        self.cve_id = cve_id
        self.cvss_score = cvss_score
        self.affected_system = affected_system
```

```
# Dataclass - same result, written once per field
@dataclass
class Vulnerability:
    cve_id: str
    cvss_score: float
    affected_system: str
```

In [ ]:
```
from dataclasses import dataclass, field
from typing import List


@dataclass
class Vulnerability:
    """Stores data about a single known vulnerability."""
    cve_id: str
    cvss_score: float
    affected_system: str
    patch_available: bool = False   # default value: most patches start unavailable


# Python auto-generates __init__ - same to use as a regular class
vuln = Vulnerability(
    cve_id="CVE-2024-1234",
    cvss_score=9.8,
    affected_system="Apache 2.4",
    patch_available=True,
)

print(vuln.cve_id)
print(vuln.cvss_score)

# Python auto-generates __repr__ - no need to write a __str__ method
print(vuln)
```

Mutable default values - the `field()` trap

Warning: You cannot use a mutable default value (like `[]` or `{}`) directly in a dataclass. Python would share the same list across all instances. Use `field(default_factory=list)` instead.

In [ ]:
```
from dataclasses import dataclass, field
from typing import List


# WRONG: would share the same list across all ScanResult instances
# @dataclass
# class ScanResult:
#     open_ports: List[int] = []   # ValueError: mutable default not allowed


# CORRECT: use field(default_factory=list) for mutable defaults
@dataclass
```

```
class ScanResult:
    """Stores the result of a port scan. open_ports defaults to an empty list."""
    target_ip: str
    analyst: str
    timestamp: str
    scan_duration_ms: int = 0
    open_ports: List[int] = field(default_factory=list)  # each instance gets its O


# Create two independent results
result_a = ScanResult(target_ip="10.0.0.1", analyst="j.smith", timestamp="2024-01-1
result_b = ScanResult(target_ip="10.0.0.2", analyst="a.jones", timestamp="2024-01-1

result_a.open_ports.append(443)
print(result_a.open_ports)  # [443]
print(result_b.open_ports)  # []  -- correctly independent
```

When dataclass falls short

Dataclasses do not support **computed attributes** natively in the field list. If you need `port_count` to always equal `len(open_ports)`, you need a `__post_init__` method.

In [ ]:
```
from dataclasses import dataclass, field
from typing import List


@dataclass
class ScanResult:
    """Scan result with a computed port_count derived in __post_init__."""
    target_ip: str
    analyst: str
    timestamp: str
    open_ports: List[int] = field(default_factory=list)
    # port_count will be set in __post_init__, not passed by the caller
    port_count: int = field(init=False)   # init=False: excluded from __init__

    def __post_init__(self):
        """Runs automatically after __init__. Sets computed fields."""
        self.port_count = len(self.open_ports)


result = ScanResult(
    target_ip="10.0.0.55",
    analyst="j.smith",
    timestamp="2024-01-15T08:30Z",
    open_ports=[22, 80, 443]
)

print(result.port_count)   # 3 - computed automatically, not passed in
print(result)              # full repr auto-generated by @dataclass
```

Practice - Section 4

Write: Create a dataclass `FirewallRule` with these fields:

- `rule_id` (str) - e.g. `"RULE-001"`
- `action` (str) - `"ALLOW"` or `"DENY"`
- `source_ip` (str)
- `destination_port` (int)
- `protocol` (str) - default `"TCP"`
- `is_active` (bool) - default `True`
- `description` (str) - a computed field (set in `__post_init__`) that formats to `"ALLOW TCP from 10.0.0.1 to :443"`

In [ ]:
```
from dataclasses import dataclass, field

# Your code here


# Test
# rule = FirewallRule("RULE-001", "ALLOW", "10.0.0.1", 443)
# print(rule.description)
# Expected: "ALLOW TCP from 10.0.0.1 to :443"
# print(rule)
```

▶ 💡 **SOLUTION**

---

5. NamedTuple - Lightweight, Immutable Records

Analogy: A NamedTuple is like a tamper-evident evidence bag. Once sealed, the contents cannot be modified. Perfect for records that should never change after creation: a logged event, a fixed network endpoint, a captured packet header.

Two ways to create a NamedTuple

Classic style (from `collections`):

```
from collections import namedtuple
Endpoint = namedtuple("Endpoint", ["ip", "port", "protocol"])
```

Typed style (from `typing`) - recommended, supports type hints:

```
from typing import NamedTuple
class Endpoint(NamedTuple):
    ip: str
    port: int
    protocol: str
```

Both produce the same kind of object. Use the typed style for cybersecurity code.

In [ ]:
```python
from typing import NamedTuple


class NetworkEndpoint(NamedTuple):
    """An immutable (ip, port, protocol) triplet. Cannot be modified after creation
    ip: str
    port: int
    protocol: str = "TCP"   # default protocol


# Create endpoints
ssh_endpoint = NetworkEndpoint(ip="10.0.0.1", port=22)
rdp_endpoint = NetworkEndpoint(ip="10.0.0.2", port=3389)
http_endpoint = NetworkEndpoint(ip="203.0.113.5", port=80, protocol="TCP")

# Access by name (like a class)
print(ssh_endpoint.ip)
print(ssh_endpoint.port)

# Access by index (like a tuple)
print(ssh_endpoint[0])    # "10.0.0.1"
print(ssh_endpoint[1])    # 22

# Unpack like a tuple
ip, port, protocol = ssh_endpoint
print(f"Endpoint: {ip}:{port} ({protocol})")
```

In [ ]:
```python
from typing import NamedTuple


class NetworkEndpoint(NamedTuple):
    ip: str
    port: int
    protocol: str = "TCP"


ssh_endpoint = NetworkEndpoint(ip="10.0.0.1", port=22)

# Immutability: you CANNOT modify a NamedTuple after creation
try:
    ssh_endpoint.port = 2222   # AttributeError
except AttributeError as e:
    print(f"Cannot modify: {e}")
# This is intentional - once an endpoint is recorded, it shouldn't change


# NamedTuples are hashable - you can use them as dict keys or store in a set
blocked_endpoints = {
    NetworkEndpoint("203.0.113.5", 4444),
    NetworkEndpoint("198.51.100.42", 1337),
}

incoming = NetworkEndpoint("203.0.113.5", 4444)
print(f"Blocked: {incoming in blocked_endpoints}")  # True
```

NamedTuple vs dataclass

Feature	NamedTuple	Dataclass
Immutable	Yes	No (by default)
Hashable (usable as dict key)	Yes	No (by default)
Computed attributes	No	Yes (`__post_init__`)
Methods	Yes (limited)	Yes (full)
Index access	Yes	No
Tuple unpacking	Yes	No
Best for	Fixed records (events, endpoints, coordinates)	Structured objects that may evolve

---

Practice - Section 5

Write: Define a `NamedTuple` called `LogEntry` with these fields:

- `timestamp` (str)
- `source_ip` (str)
- `action` (str) - e.g. `"DENY"` or `"ALLOW"`
- `destination_port` (int)

Then create a list of 3 `LogEntry` objects representing firewall log lines, and print each one using tuple unpacking.

In [ ]:
```
from typing import NamedTuple, List

# Your code here
```

▶ 💡 **SOLUTION**

---

6. Combining Structures

Analogy: A SOC analyst doesn't choose between sticky notes, forms, or evidence bags - they use all three for different parts of an investigation. Python data structures work the same way. Combine them where each fits best.

The pattern:

- Use **NamedTuple** for small, fixed, immutable sub-records (an endpoint, a coordinate, a CVE reference)
- Use **class** or **dataclass** for the main structured object that owns those sub-records
- Use a **dict** only at the outer edge where you need JSON serialisation or dynamic keys

In [ ]:
```
from dataclasses import dataclass, field
from typing import List, NamedTuple


# NamedTuple for the small, immutable sub-record
class PortFinding(NamedTuple):
    """An immutable record of a single open port finding."""
    port: int
    service: str
    is_dangerous: bool


# Dataclass for the main scan result object
@dataclass
class ScanReport:
    """Full scan report for one target host, containing a list of PortFinding recor
    target_ip: str
    analyst: str
    timestamp: str
    findings: List[PortFinding] = field(default_factory=list)
    # Computed
    dangerous_count: int = field(init=False)

    def __post_init__(self):
        self.dangerous_count = sum(1 for f in self.findings if f.is_dangerous)

    def summary(self) -> str:
        risk = "HIGH RISK" if self.dangerous_count > 0 else "CLEAN"
        return (
            f"{risk} | {self.target_ip} | "
            f"{len(self.findings)} port(s) found, "
            f"{self.dangerous_count} dangerous"
        )


# Build the findings using NamedTuple
findings = [
    PortFinding(port=22,   service="SSH",   is_dangerous=False),
    PortFinding(port=3389, service="RDP",   is_dangerous=True),
    PortFinding(port=445,  service="SMB",   is_dangerous=True),
    PortFinding(port=443,  service="HTTPS", is_dangerous=False),
]

# Build the main report using Dataclass
report = ScanReport(
    target_ip="10.0.0.55",
    analyst="j.smith",
    timestamp="2024-01-15T08:30Z",
    findings=findings,
```

```
)

print(report.summary())
print(f"\nDetailed findings for {report.target_ip}:")
for f in report.findings:
    risk_label = "[!!!]" if f.is_dangerous else "[ ok]"
    print(f"  {risk_label} Port {f.port:5d} - {f.service}")
```

The type hint complexity rule

A good indicator that you need a better data structure: look at your type hints. If you see this:

`scan_data: Dict[str, List[Dict[str, Dict[str, int]]]] = ...`
...you have nested dicts inside dicts inside lists inside dicts. This is a signal to create a proper class or NamedTuple and name your concepts. Compare:

```
# Before: unreadable
def process(data: Dict[str, List[Dict[str, Dict[str, int]]]]) -> None: ...

# After: readable
def process(report: ScanReport) -> None: ...
```

Practice - Section 6

Write: Model a vulnerability report using combined structures:

1. A `NamedTuple` called `CVERef` with fields `cve_id` (str) and `cvss_score` (float)
2. A `@dataclass` called `VulnerabilityReport` with:
   - `target_ip` (str)
   - `analyst` (str)
   - `cve_list` (List[CVERef]) - defaults to empty list
   - `critical_count` (int) - computed in `__post_init__` : number of CVEs with score >= 9.0
3. Instantiate the report with at least 3 CVEs (mix of critical and non-critical)
4. Print a summary showing the host IP and how many critical vulnerabilities were found

In [ ]:
```
from dataclasses import dataclass, field
from typing import List, NamedTuple

# Your code here
```

▶ 💡 **SOLUTION**

---

7. When to Use Which - Decision Guide

```
Does the data need to change after creation?
  NO  -> Is it small (2-5 fields) with no behaviour?
            YES -> NamedTuple
            NO  -> Dataclass with frozen=True
  YES -> Does it have computed attributes or complex behaviour?
            YES -> Regular Class
            NO  -> Dataclass

Is it a quick throwaway script or needs to be JSON?
  YES -> Dict
```

Structure	Mutable	Computed attrs	Behaviour	Hashable	Boilerplate
`dict`	Yes	No	No	No	Very low
`class`	Yes	Yes	Yes	No	High
`@dataclass`	Yes	Via `__post_init__`	Yes	No	Low
`NamedTuple`	No	No	Limited	Yes	Very low

Security-domain mapping

What you're modelling	Recommended structure
A network endpoint (ip, port, protocol)	`NamedTuple` - fixed, immutable, hashable
A raw log line parsed from file	`NamedTuple` - read-only record
A CVE reference with score	`NamedTuple` - small, fixed
A scan result with computed risk level	`@dataclass` with `__post_init__`
A firewall ruleset that evolves	Regular `class`
A SOC alert that gets acknowledged	Regular `class` (mutable state + methods)
Quick parse of an unknown config file	`dict`
JSON to send to a SIEM API	`dict`

---

8. Summary

Structure	Import	Key syntax	Best for
`dict`	None	`{"key": value}`	Quick scripts, JSON, dynamic schemas
`class`	None	`def __init__(self, ...)`	Full control, complex behaviour, mutable state

Structure	Import	Key syntax	Best for
`@dataclass`	`from dataclasses import dataclass, field`	`field: type = default`	Structured objects, less boilerplate
`NamedTuple`	`from typing import NamedTuple`	`class X(NamedTuple): field: type`	Small, immutable, hashable records

Key rules to remember

1. **99% of the time, combine structures** - NamedTuple for sub-records, class/dataclass for the parent object.
2. **Watch your type hint complexity** - `Dict[str, List[Dict[str, int]]]` is a smell; name your concepts instead.
3. **Use `field(default_factory=list)`** for mutable defaults in dataclasses - never `= []`.
4. **Use `field(init=False)` + `__post_init__`** for computed attributes in dataclasses.
5. **NamedTuples are hashable** - you can store them in sets and use them as dict keys.
6. **Dicts map directly to JSON** - use them at API/serialisation boundaries, not as your main data model.

Next Steps

- Complete **01_Drills_Data_Structures.ipynb** - 15 exercises on all four structures
- Optional: look into `pydantic` (third-party library) - it extends dataclasses with runtime type validation, widely used in security APIs